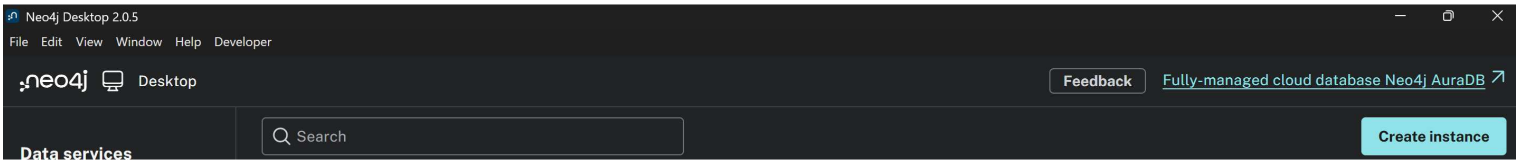
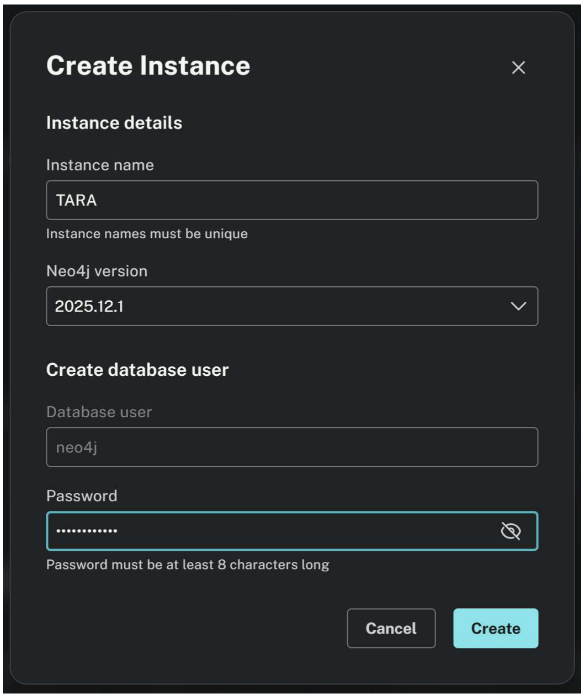
Database Import Instructions (Neo4j)

1:- Create a database Instance



To import the database into **Neo4j**, first create a new database instance and assign it a name of your choice. Click **Create Instance** to proceed. A screen similar to the one shown below will appear.



The default username is **neo4j**, and the password must be set to **comealongway**, as the Python-based generator depends on these credentials.

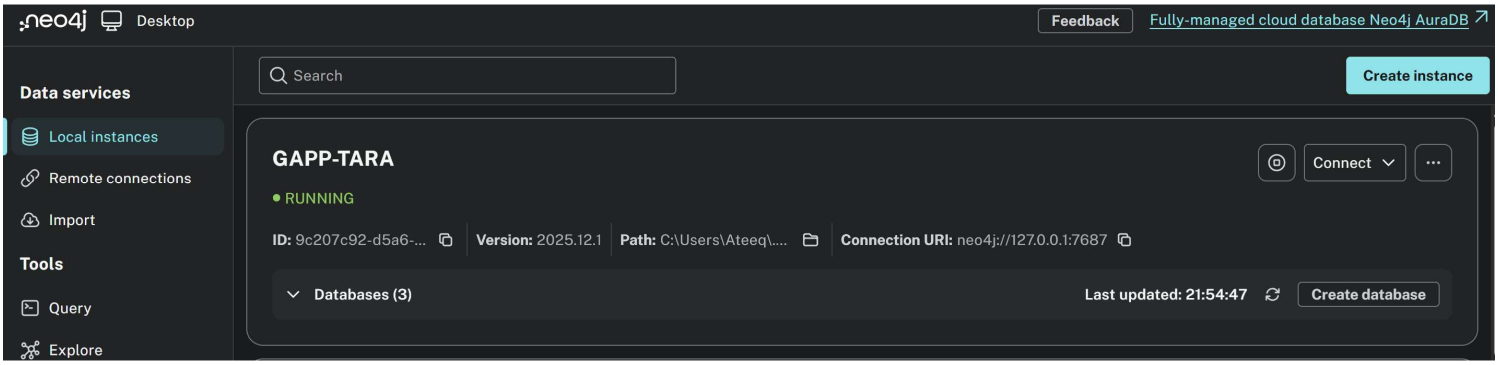
After configuring the credentials, click **Start** to launch the database.

Troubleshooting

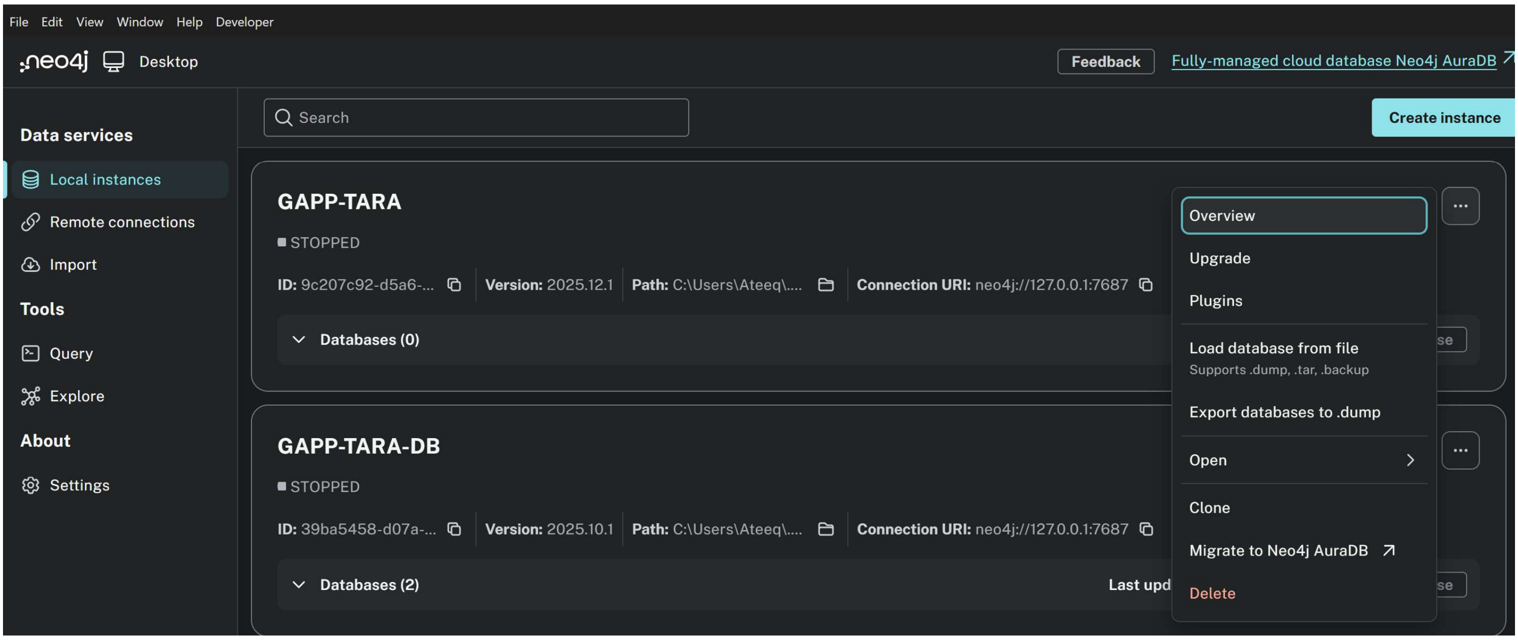
If the database does not start successfully:

1. Re-enter the same username and password and try starting the database again.
2. If the problem persists, open the `table6_generator.py` file and update the password to a value of your choice.
3. Ensure that the updated password in `table6_generator.py` matches the password configured in Neo4j, then restart the database.

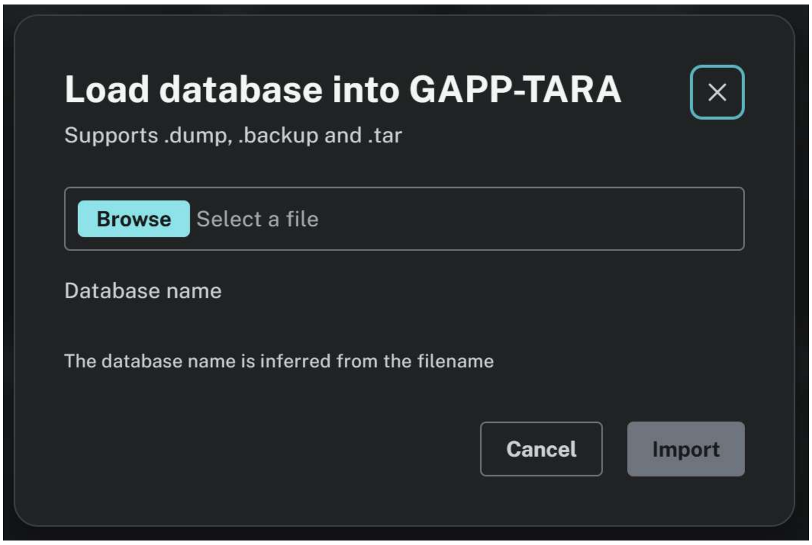
Once the database starts successfully, you should see a screen similar to the one shown below.



2 Import the database: - Click the three dots on the right-hand side and select **"Load database from file."** Then, locate the relevant file and upload the dump file, which contains the actual database.

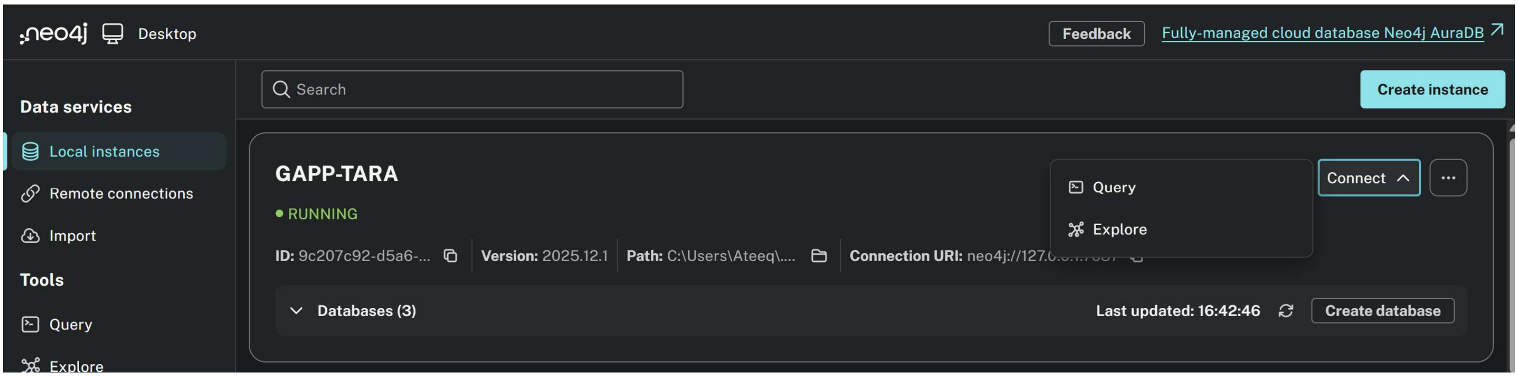


3: You will be the directed to a new screen.

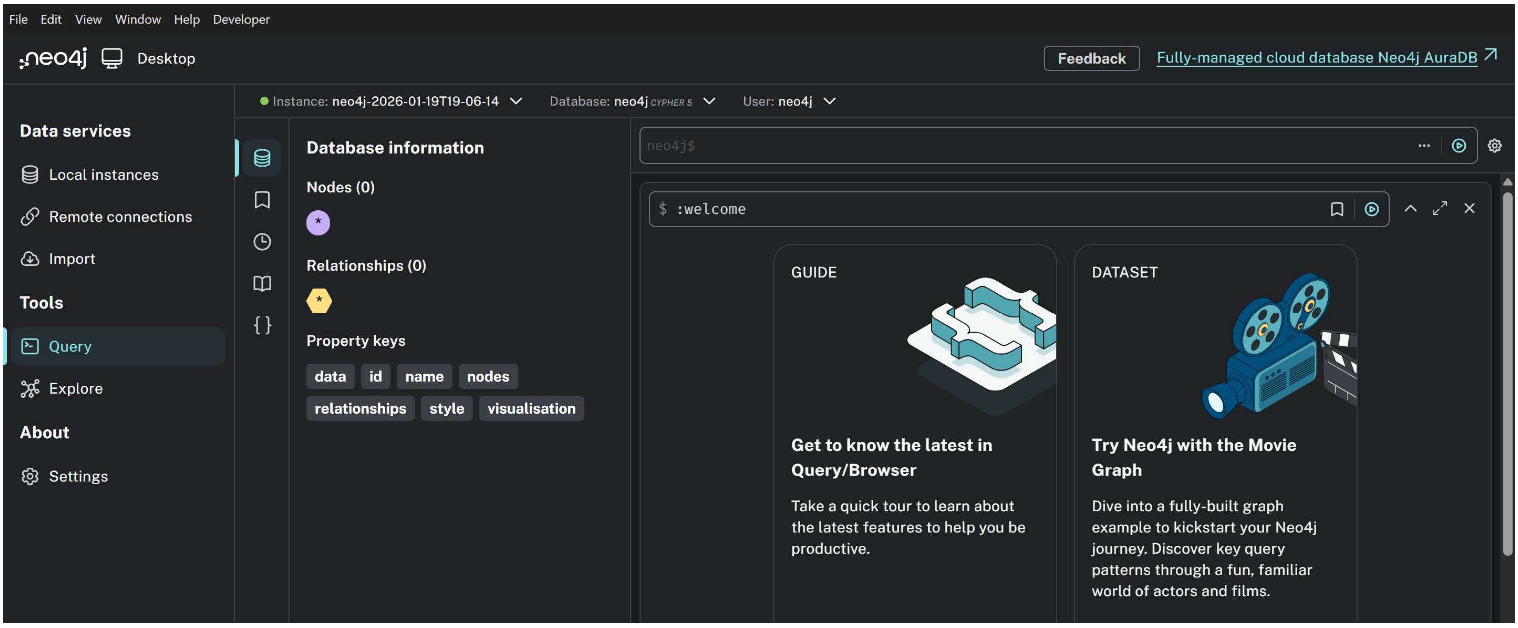


Click the **Browse** button, locate the database dump file (**`neo4j-2026-01-19T19-06-14`** in our case), and select it to proceed with browsing and importing the database.

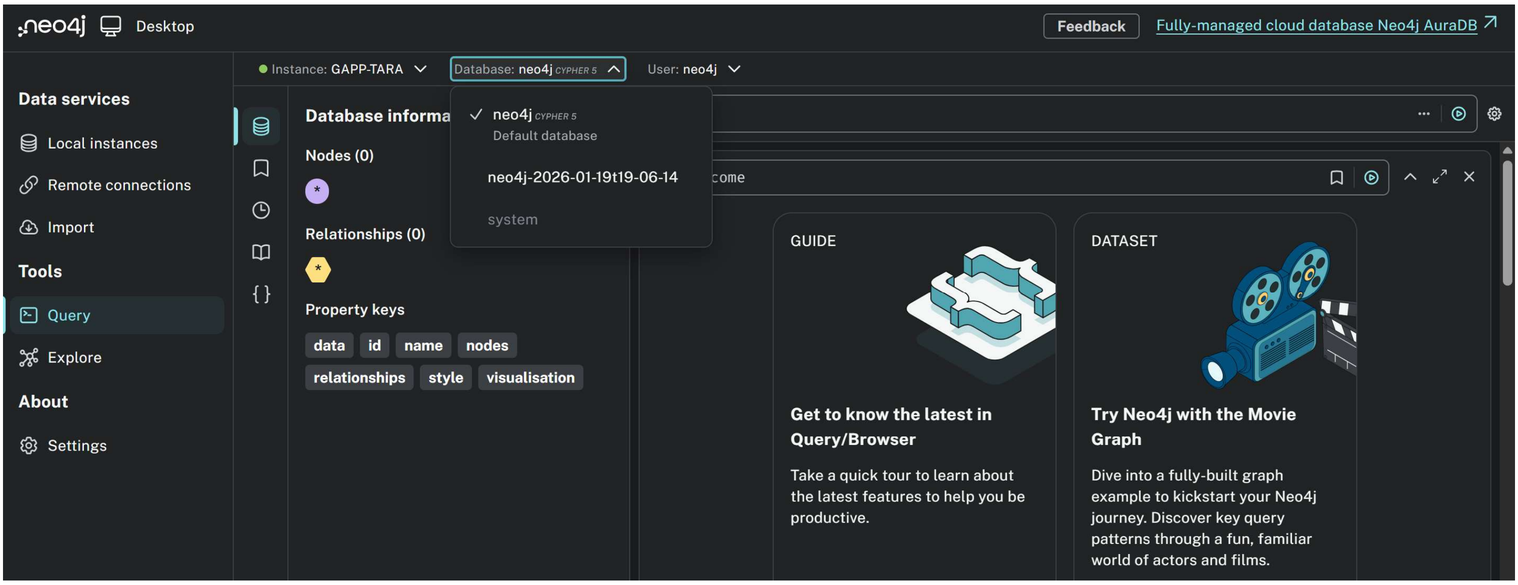
4 Connect Query:- Click the **connect** button and select Query



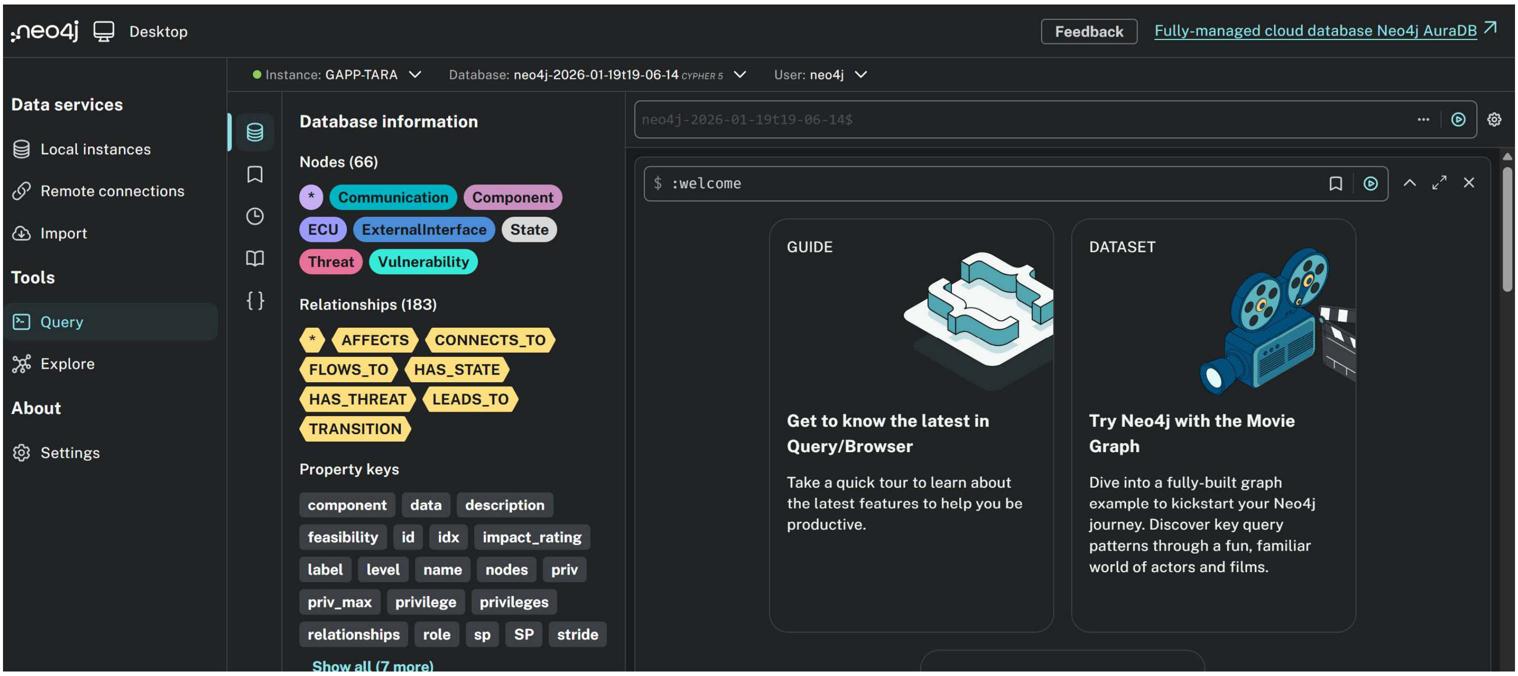
5:- You will be presented with a screen similar to the following.



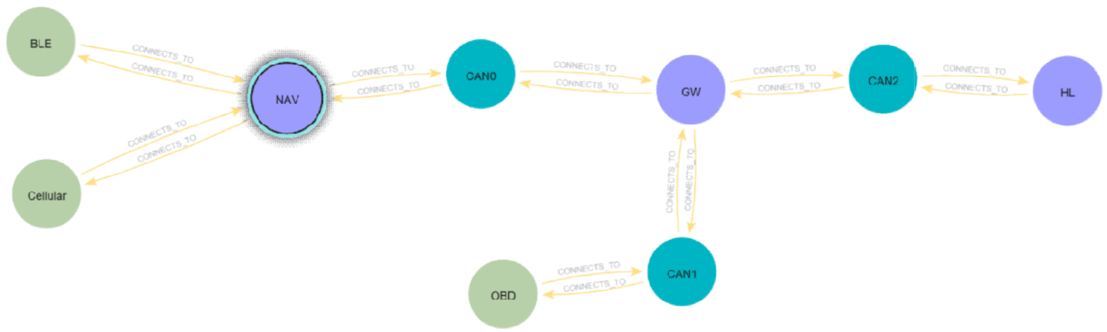
By default, Neo4j selects the **neo4j** database. Please ensure that you switch the active database to the imported one, **neo4j-2026-01-19T19-06-14**, before proceeding.



Once you have selected `neo4j-2026-01-19T19-06-14` as the active database, the interface should update accordingly. At this point, you should see a screen similar to the one shown below.



Click on `CONNECTS_TO`, after which the following graph will be displayed. (Recreation of model from the reference paper)



Which closely resembles the graph presented in the reference paper.

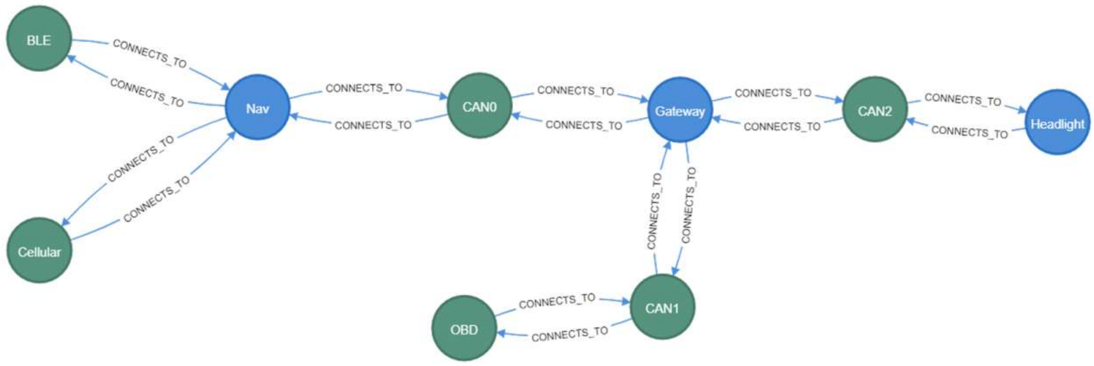
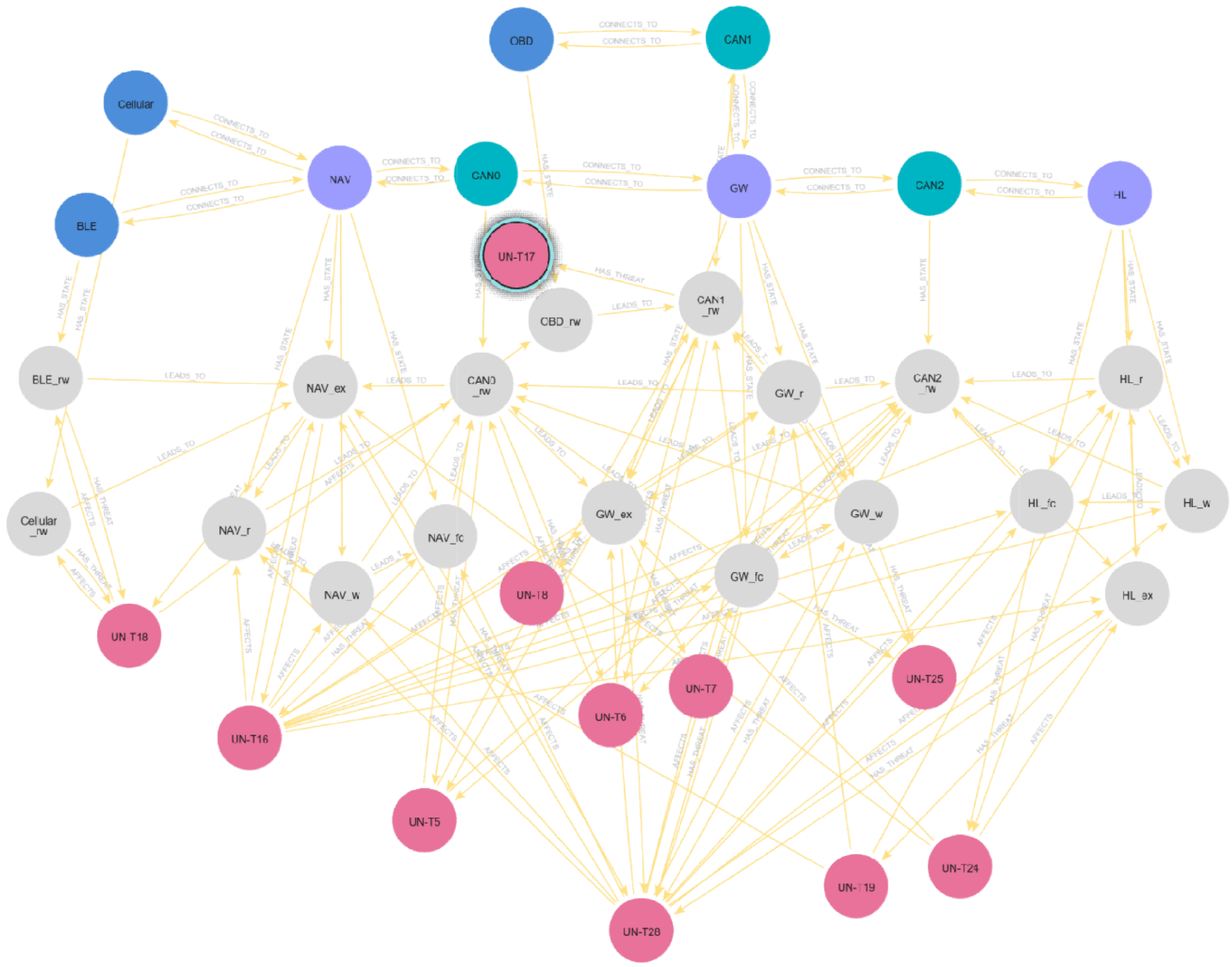


Figure 6. Implementation of the GAPP system model in Neo4j graph database. (From reference paper, Graph-Based Automation of Threat Analysis and Risk Assessment for Automotive Security by Mera Nizam-Edden Saulaiman, Miklos Kozlovszky, and AkosCsilling)

Run the following Cypher query in the **Neo4j Browser** to display the second graph.

```
MATCH (n)
OPTIONAL MATCH (n)-[r]->(m)
RETURN n, r, m;
```



which closely resembles the graph presented in the reference paper.

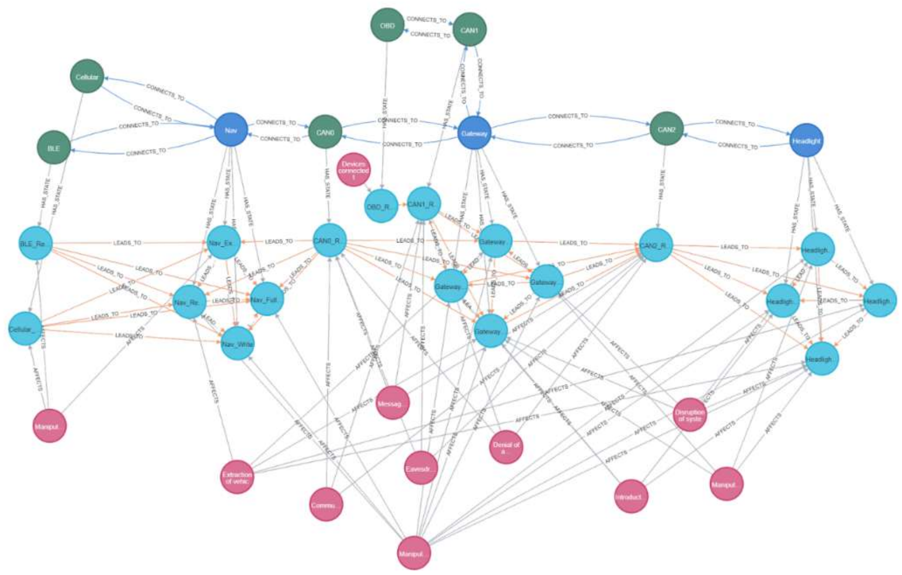


Figure 8. The full graph implementation of the example system in Section 5.1. Vulnerabilities are mapped to state nodes, which are mapped to architecture elements.(from reference paper, Graph-Based Automation of Threat Analysis and Risk Assessment for Automotive Security by Mera Nizam-Edden Saulaiman, Miklos Kozlovszky, and AkosCsilling)

Launch Scenario 2

```
MATCH (a:Component)-[r:FLOWS_TO]->(b:Component)
WHERE a.name IN ['BLE','Cellular','TCU','GW','CAN_Body','DoorLock']
  AND b.name IN ['BLE','Cellular','TCU','GW','CAN_Body','DoorLock']
RETURN a, r, b;
```